Creating a Table with SQL

Lecture Notes · Game DB Junior

In this lesson

- What **keywords** and **identifiers** are
- What each line of the **CREATE TABLE** statement does
- Two column data types: **VARCHAR(50)** and **INTEGER**

1. Our first SQL

We wrote our first piece of SQL to create a table inside our database. Now we look deeper at this SQL and try to understand its **syntax**.

```
CREATE TABLE cities (
    name VARCHAR(50),
    country VARCHAR(50),
    population INTEGER,
    area INTEGER
);
```

2. Keywords and identifiers

This SQL statement is made of different words. Some words are **keywords**. Other words are **identifiers**.

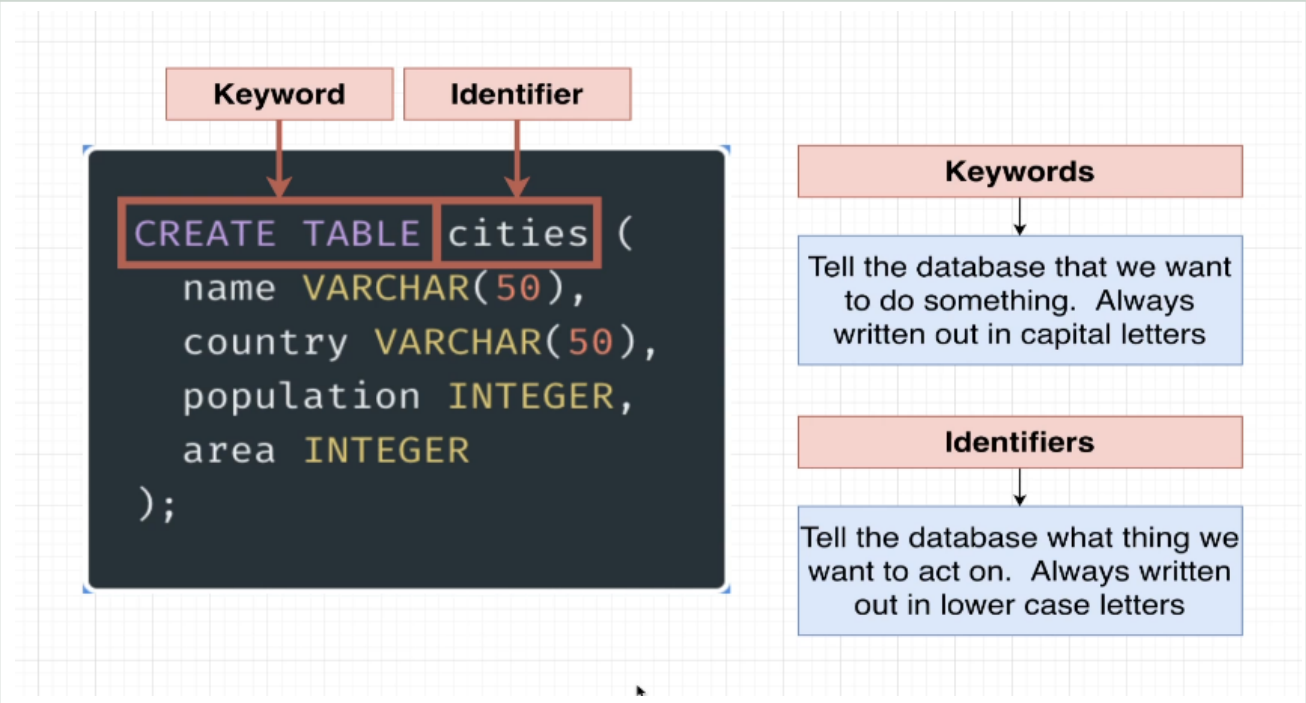


Figure 1 — Keywords and identifiers

	Keywords	Identifiers
What they do	Tell the database that we want to do a specific thing, or choose an option	Tell the database what thing we want to act on
How we write them	Always in **CAPITAL** letters	Always in **lower case** letters
Example	CREATE TABLE	cities

3. What each line does

The statement goes over many lines. Each line has its own purpose in creating the cities table.

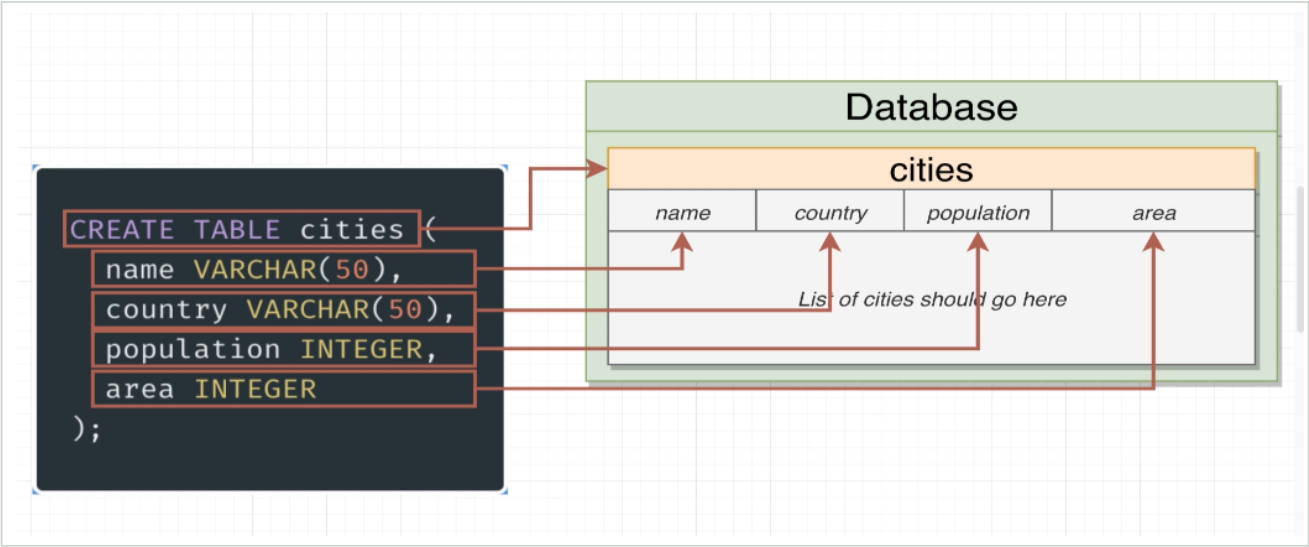


Figure 2 — Each line of the SQL and what it creates in the database

Line	What it does
CREATE TABLE cities (	Creates the table itself
name VARCHAR(50),	Creates the **name** column. It stores data of type VARCHAR(50)
country VARCHAR(50),	Creates the **country** column. It stores data of type VARCHAR(50)
population INTEGER,	Creates the **population** column. It stores data of type INTEGER
area INTEGER	Creates the **area** column. It stores data of type INTEGER

- Inside the **parentheses ( )** we list all the columns the table should have. Here: four columns.
- Right after each column name, we write the **type of data** that column will store.

4. Column data types

VARCHAR(50) and INTEGER are two examples of **column data types**.

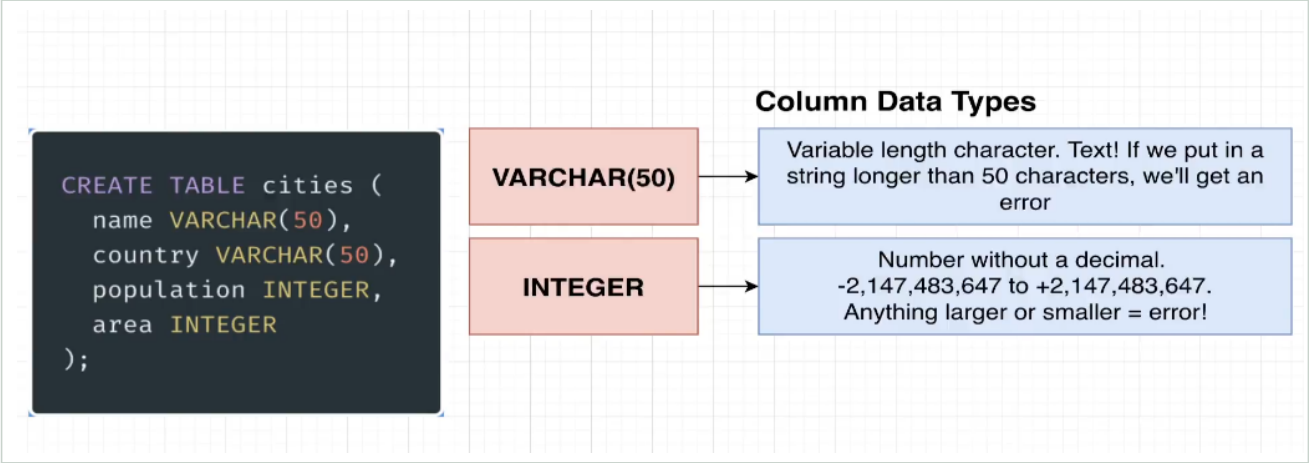


Figure 3 — Column data types

VARCHAR(50)

- Short for **variable character**. You can think of it as a **string** (text).
- The **50** means we never expect to store a string longer than 50 characters.
- If we try to insert a name or a country longer than 50 characters, Postgres gives us an **error**.
- So the number only limits the length of the string we can put into this field.

INTEGER

- In Postgres, integers are **numbers without decimals**.
- They go from **-2,147,483,647** to **+2,147,483,647** (about -2 billion to about +2 billion).
- If we try to insert a larger or smaller value into population or area, we get an **error**.

Is INTEGER big enough for cities? We have to ask: will a city ever have a population or an area greater than 2 billion? Right now Tokyo is the largest city, with about 38 million people. So we do not expect a city with more than 2 billion people any time soon. INTEGER is fine for this example.

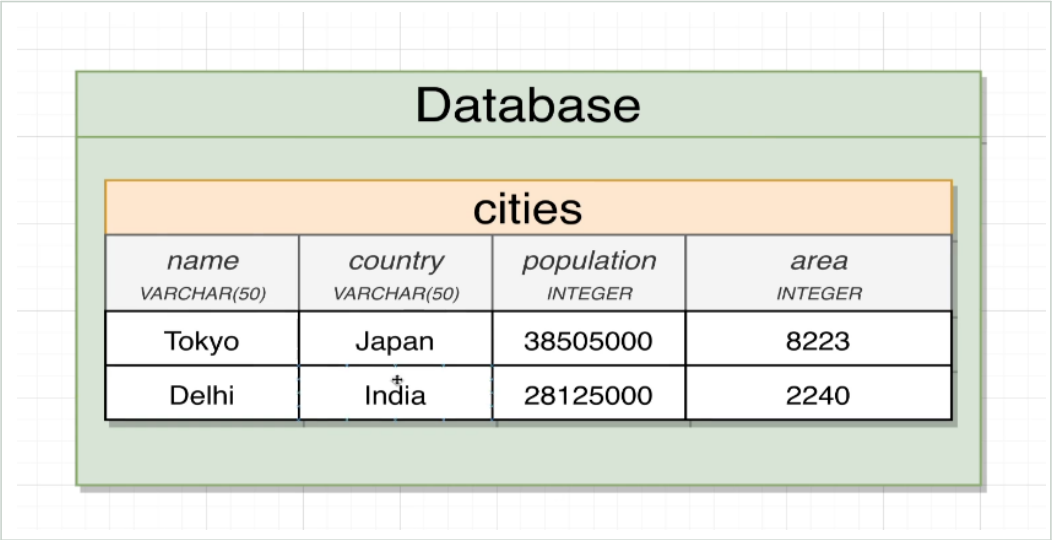


Figure 4 — The cities table with a data type for each column

5. Running the code (my practice)

I wrote the same SQL with short comments. In SQL, a comment starts with **--**.

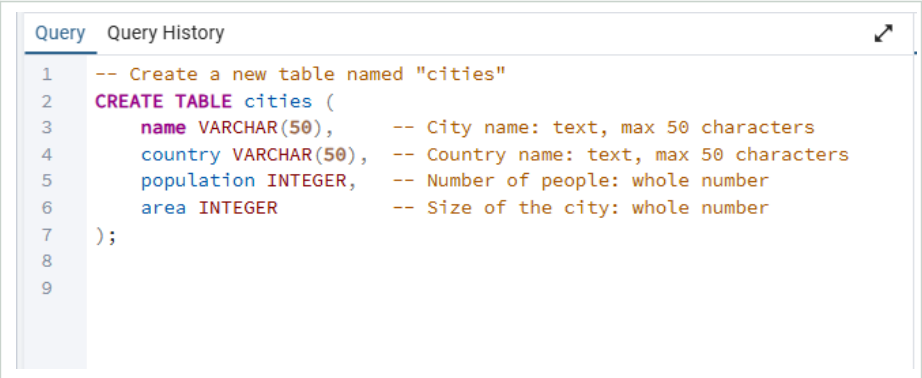


Figure 5 — The CREATE TABLE statement with comments

After running it, the Messages tab shows **CREATE TABLE** and "Query returned successfully".



Figure 6 — The query ran successfully

Now the cities table is in the database with its 4 columns. Here VARCHAR(50) is shown as **character varying(50)**.

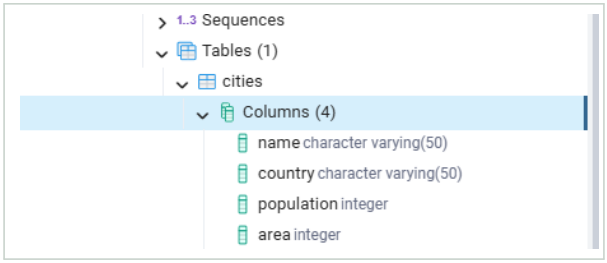


Figure 7 — The cities table and its 4 columns

6. Summary

Word	Meaning	Example
Keyword	Tells the database to do something. CAPITAL letters	CREATE TABLE
Identifier	Tells the database what to act on. lower case letters	cities
VARCHAR(50)	Text, max 50 characters. Longer = error	name, country

Word	Meaning	Example
INTEGER	Number without a decimal, about -2 billion to +2 billion. Outside = error	population, area

We will see many more data types later, but VARCHAR and INTEGER are a good place to start. **Next lesson:** how to insert data into a table.

Sources

Lecture transcript and lecture images (Figures 1–4)
My own practice screenshots (Figures 5–7)